

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра «Информатика»

Лабораторная работа №4
**«Разработка консольных проектов Visual Studio с
использованием функций VC++»**
по Разделу
**«Функции VC++ и консольные проекты
Visual Studio»**
по дисциплине
«Введение в информационные технологии»

Вариант 1

Выполнил: студент гр. ЗРС2502 Баранова Е.В

Проверил: Рюмин Ю.А.

Москва, 2025 г.

1) Общее задание на разработку программного проекта

1) Изучите структуру программного кода консольных проектов Visual Studio и правила определения, объявления и вызова функций VC++.

2) Выберите индивидуальный вариант задания из таблицы 3.1.

3) Решите задачу вычисления заданного арифметического выражения с использованием функций VC++ (без использования функций она уже решена в предыдущей работе 2). Для этого разработайте три варианта схем алгоритмов и соответствующих процедур, реализующих решения задачи:

- схемы алгоритмов для вычисления заданного арифметического выражения:
 - схему алгоритма процедуры с входными параметрами и возвращаемым значением;
 - схему алгоритма процедуры с входными и выходными параметрами и без возвращаемого значения;
 - схему алгоритма без параметров и без возвращаемого значения;
- программные коды трех функций и функции main в соответствии со схемами алгоритмов.

4) Создайте консольное решение, содержащее пять проектов, каждый из которых содержит одну из разработанных функций п.3 и главную функцию main, в которой осуществляется ввод исходных данных, вызов соответствующей функции п.3 и вывод результата:

- функция с параметрами и возвращаемым значением, причем определение функции должно быть записано перед функцией main.
- функция с параметрами и возвращаемым значением, причем определение функции должно быть записано после функции main.
- функция с параметрами и без возвращаемого значения.
- функция без параметров и без возвращаемого значения (с глобальными переменными).
- функция с параметрами и возвращаемым значением, причем определение функции и main должны находиться в разных файлах.

Каждый способ должен быть реализован в отдельном проекте, а все пять проектов должны быть объединены в одном решении.

5) Выполните созданные проекты и получите результаты. Убедитесь в идентичности и правильности результатов, полученных при выполнении каждого из пяти проектов.

6) Проведите эксперименты, описанные в примере выполнения задания. Внесение изменений в программный код выполняйте путем комментирования исходного кода с последующим удалением комментария для возврата к исходному состоянию. После внесения изменений выполняйте повторную компиляцию и перестроение решения

2) Индивидуальное задание на разработку программного проекта

Создать решение (например, с именем lab5), состоящее из пяти программных проектов, для вычисления арифметического выражения:

$$t = \cos \frac{\pi}{7} * \frac{\sin^2(x-8y)}{2,7(x-\pi)} \quad (1)$$

при значениях исходных данных $x=3,59$ и $y=17,53$ с использованием различных способов обмена данными и местоположения функций в соответствии с общим заданием.

3) Формализация и уточнение задания

Для формализации и уточнения задания определим, что исходные данные x , y – вещественного типа `double`. Результаты вычислений – переменная t также должна быть вещественного типа `double`. Операция вычисления t будет записываться следующим оператором VC++:

`t=cos(M_PI / 7)*(pow(sin(x - 8 * y),2) / 2,7 *(x - M_PI));`

Вычисление t реализуем в функциях VC++ тремя различными способами в соответствии с общим заданием.

4) Разработка пяти программных проектов в одном решении и получение результатов их работы

Создадим пять проектов в одном решении. Для этого при создании первого проекта даем проекту и решению разные имена. Решению дали имя

lab4, а проекту – имя Project1, см. рисунок 1. Далее будем добавлять проекты в уже существующее решение.

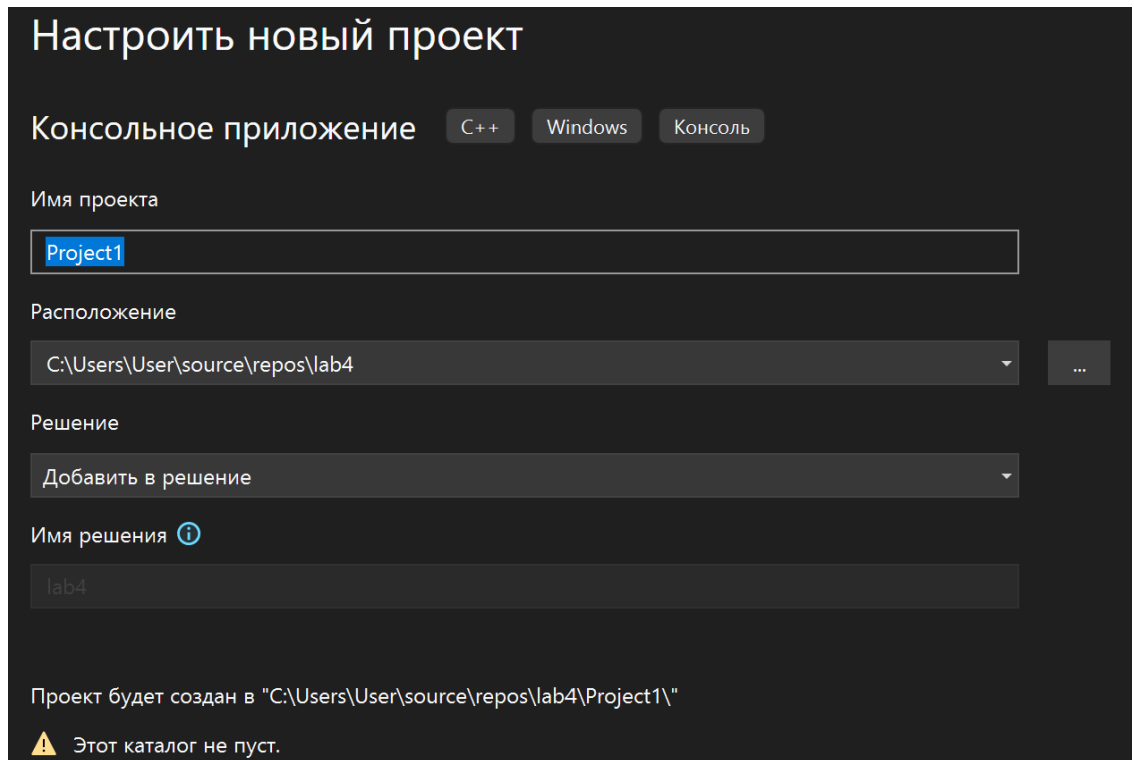


Рисунок 1. – Создание проекта и решения.

Реализация 1-го проекта

Алгоритм главной процедуры не зависит от способа обмена данными и приведен на рисунке 2.

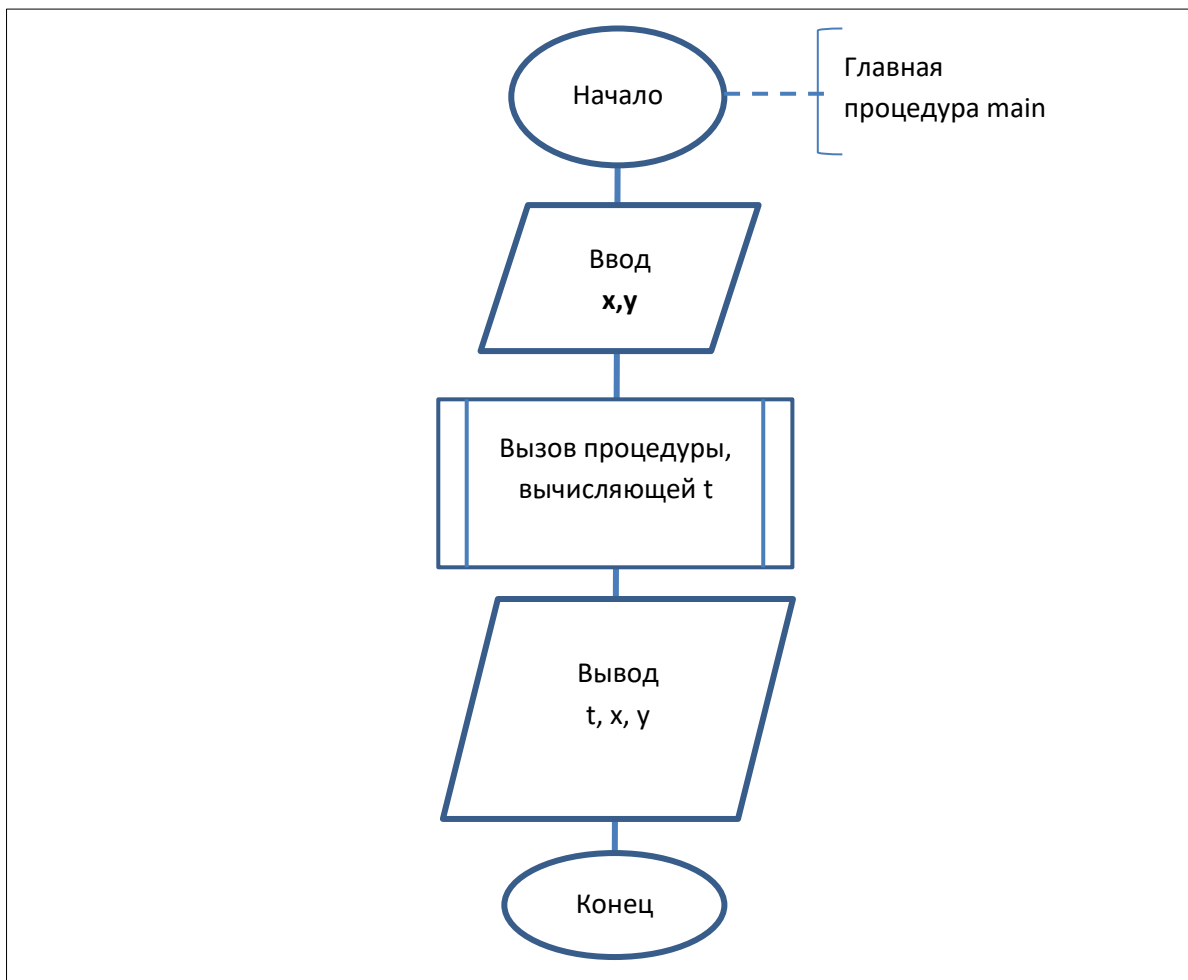


Рисунок 2. – Схема алгоритма главной процедуры main для всех проектов

Схема алгоритма процедуры func1 с параметрами и возвращаемым значением представлена на рисунке 3.

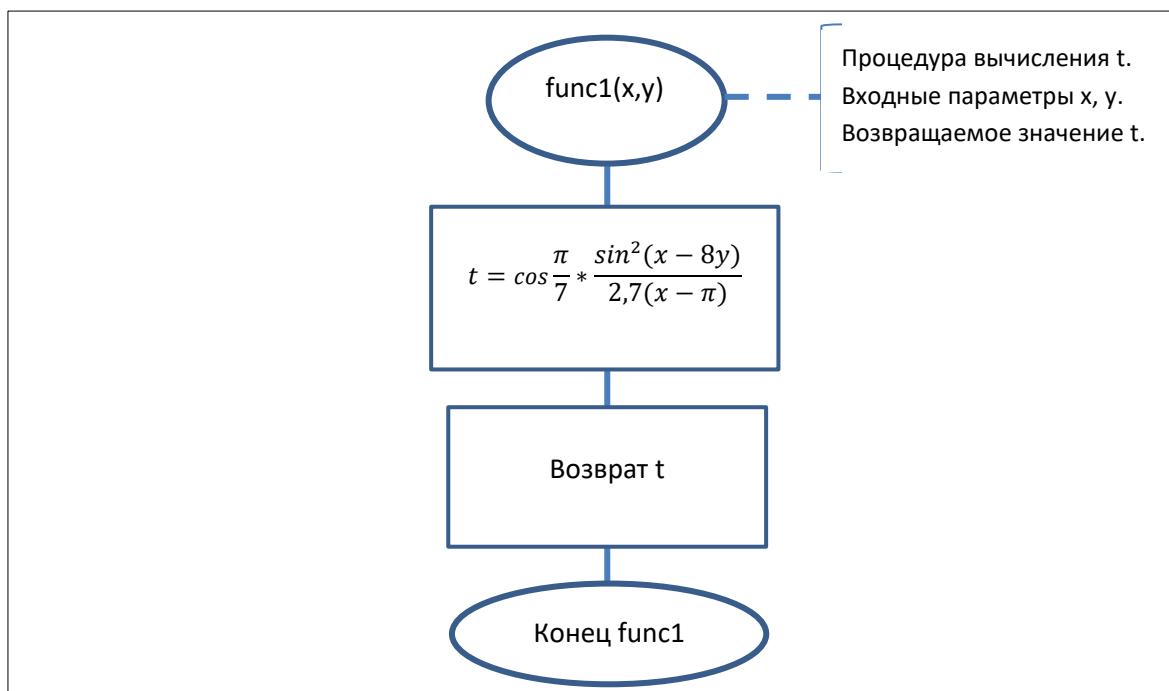
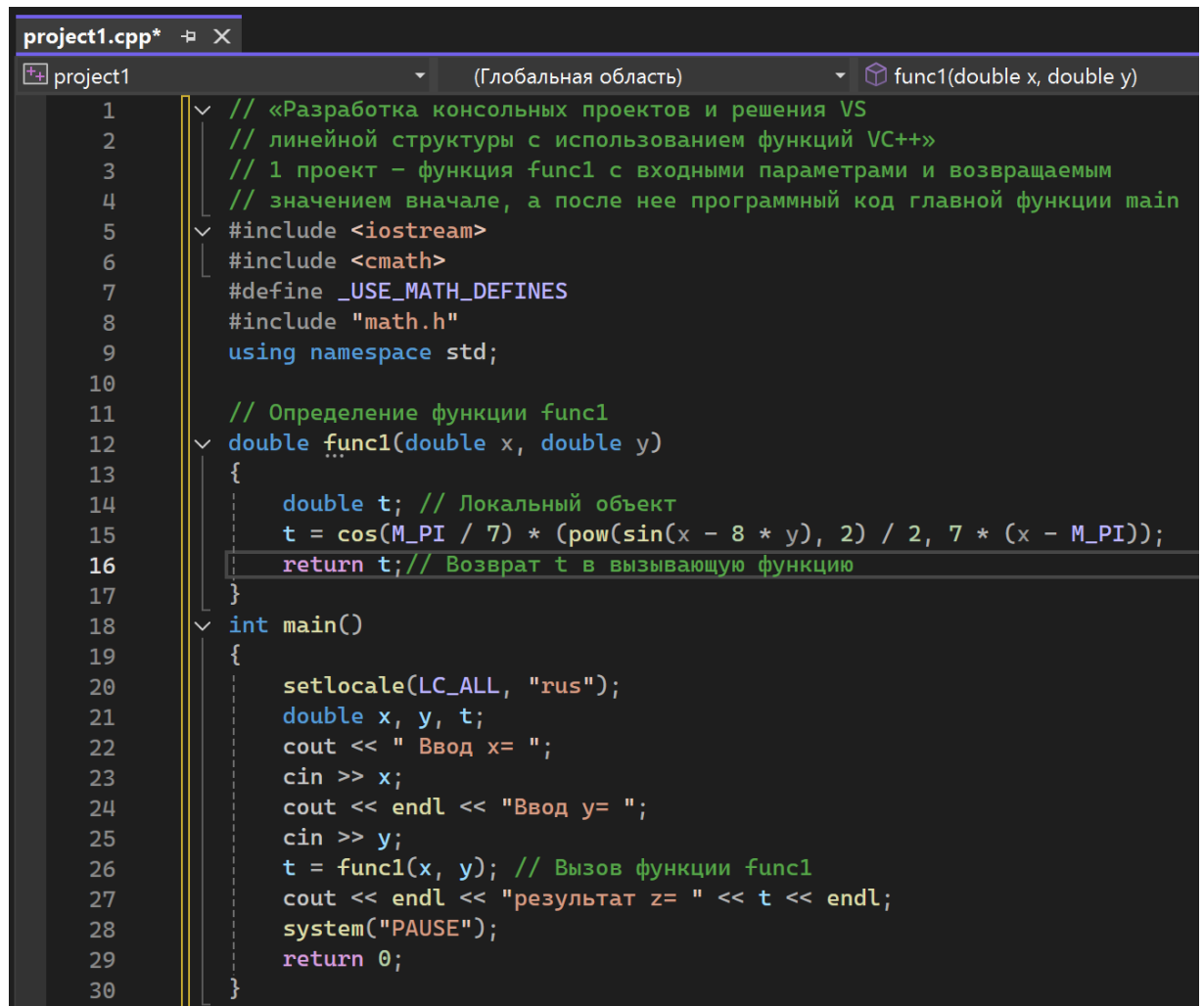


Рисунок 3 - Схема алгоритма процедуры func1 с параметрами и возвращаемым значением для первого проекта

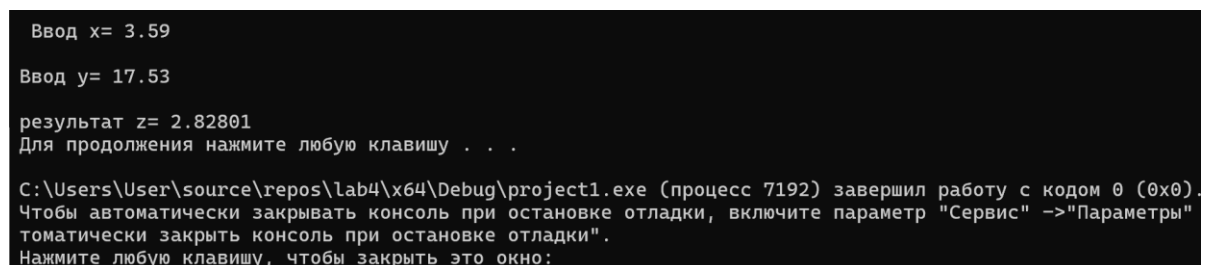
Программный код первого проекта можно увидеть на рисунке 4.



```
project1.cpp*  X
project1      (Глобальная область)  func1(double x, double y)
1  // «Разработка консольных проектов и решения VS
2  // линейной структуры с использованием функций VC++»
3  // 1 проект – функция func1 с входными параметрами и возвращаемым
4  // значением вначале, а после нее программный код главной функции main
5  #include <iostream>
6  #include <cmath>
7  #define _USE_MATH_DEFINES
8  #include "math.h"
9  using namespace std;
10
11  // Определение функции func1
12  double func1(double x, double y)
13  {
14      double t; // Локальный объект
15      t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
16      return t; // Возврат t в вызывающую функцию
17  }
18  int main()
19  {
20      setlocale(LC_ALL, "rus");
21      double x, y, t;
22      cout << " Ввод x= ";
23      cin >> x;
24      cout << endl << "Ввод y= ";
25      cin >> y;
26      t = func1(x, y); // Вызов функции func1
27      cout << endl << "результат z= " << t << endl;
28      system("PAUSE");
29      return 0;
30  }
```

Рисунок 4. – программный код первого проекта

Откомпилируем файл zad1.cpp, выполним построение решения lab5 и выполнение проекта pr1. Получим следующие результаты при заданных значениях исходных данных (рисунок 5).



```
Ввод x= 3.59
Ввод y= 17.53
результат z= 2.82801
Для продолжения нажмите любую клавишу . . .

C:\Users\User\source\repos\lab4\x64\Debug\project1.exe (процесс 7192) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры"
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 5. - Результаты выполнения проекта pr1

Проведем следующие эксперименты, используя при необходимости пошаговую отладку:

- проверим, зависит ли результат выполнения проекта от порядка фактических параметров в функции main, заменив оператор вызова функции $z=\text{func1}(x, y)$ на оператор $z=\text{func1}(y, x)$.

Результат выполнения проекта изменился, так как переменная x используется вместо переменной y и наоборот, см. рисунок 6

```
project1 (Глобальная область) main()
13 {
14     double t; // Локальный объект
15     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
16     return t; // Возврат t в вызывающую функцию
17 }
18 int main()
19 {
20     setlocale(LC_ALL, "rus");
21     double x, y, t;
22     cout << " Ввод x= ";
23     cin >> x;
24     cout << endl << "Ввод y= ";
25     cin >> y;
26     t = func1(y, x); // Вызов функции func1
27     cout << endl << "результат z= " << t << endl;
28     system("PAUSE");
29     return 0;
30 }
31
```

Консоль отладки Microsoft Visual Studio

Ввод x= 3.59

Ввод y= 17.53

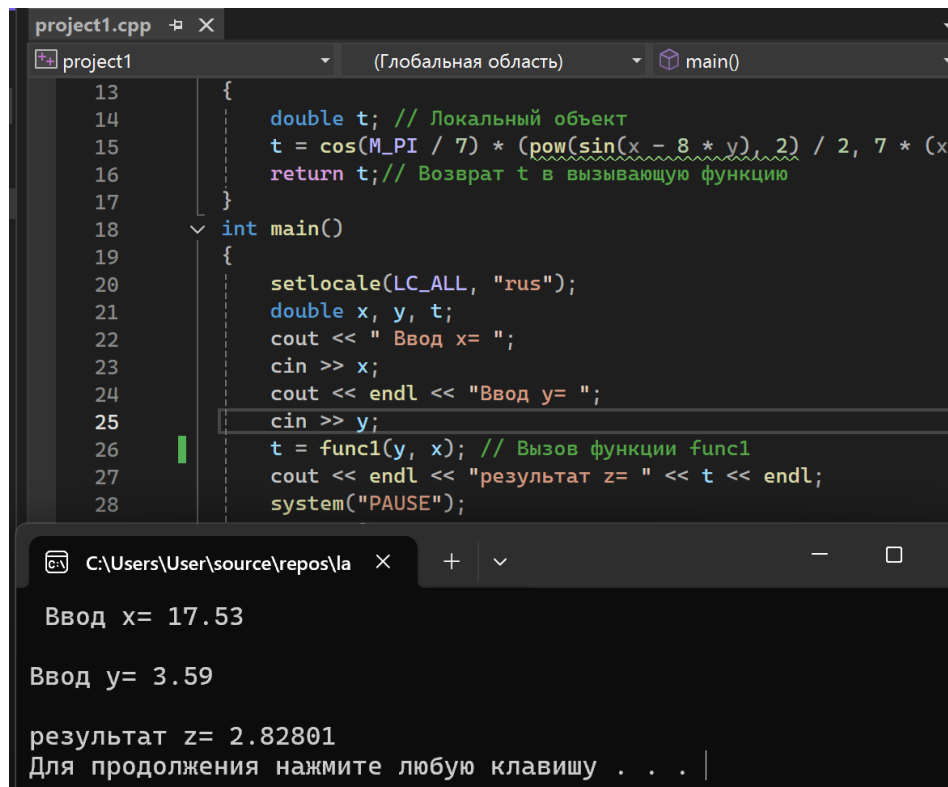
результат z= 90.7445

Для продолжения нажмите любую клавишу . . .

C:\Users\User\source\repos\lab4\x64\Debug\project1.exe (процесс 2712) завершил работу с кодом 0 (0x0).

Рисунок 6. – Изменение результата в зависимости от порядка параметров функции.

Подтверждаем нашу гипотезу меняя значения переменных, и получаем корректный результат, см. рисунок 7.



```
project1.cpp X
project1 (Глобальная область) main()
13 {
14     double t; // Локальный объект
15     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x
16     return t; // Возврат t в вызывающую функцию
17 }
18 int main()
19 {
20     setlocale(LC_ALL, "rus");
21     double x, y, t;
22     cout << " Ввод x= ";
23     cin >> x;
24     cout << endl << "Ввод y= ";
25     cin >> y;
26     t = func1(y, x); // Вызов функции func1
27     cout << endl << "результат z= " << t << endl;
28     system("PAUSE");
}

C:\Users\User\source\repos\la X + - □
Ввод x= 17.53
Ввод y= 3.59
результат z= 2.82801
Для продолжения нажмите любую клавишу . . . |
```

Рисунок 7. – корректный результат при изменении значений переменных.

- проверим, изменится ли значение переменной x в функции main, если внутри функции func1 до оператора return z изменить значение x, например, добавив оператор x++:

Результат выполнения программы не изменился(см. рисунок 8), так как значение переменной x было изменено уже после получения t. Кроме того, исходя из добавленного в конце вывода x, значение переменной x в main не изменилось, так как в функции func1 хотя и используются переменные с теми же именами, это другие переменные, и мы не выводим их из функции в отличие от t. Все что было в функции остается в функции. Это соотносится с результатом первого эксперимента с перепутанными именами переменных.


```
12  double func1(double x, double y)
13  {
14      double t; // Локальный объект
15      t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
16      x++;
17      return t; // Возврат t в вызывающую функцию
18  }
19  int main()
20  {
21      setlocale(LC_ALL, "rus");
22      double x, y, t;
23      cout << " Ввод x= ";
24      cin >> x;
25      cout << endl << "Ввод y= ";
26      cin >> y;
27      t = func1(x, y); // Вызов функции func1
28      cout << endl << "результат z= " << t << endl;
29      cout << x << endl;
30      system("PAUSE");
31      return 0;
32  }
33
```

Консоль отладки Microsoft Vi

Ввод x= 3.59

Ввод y= 17.53

результат z= 2.82801

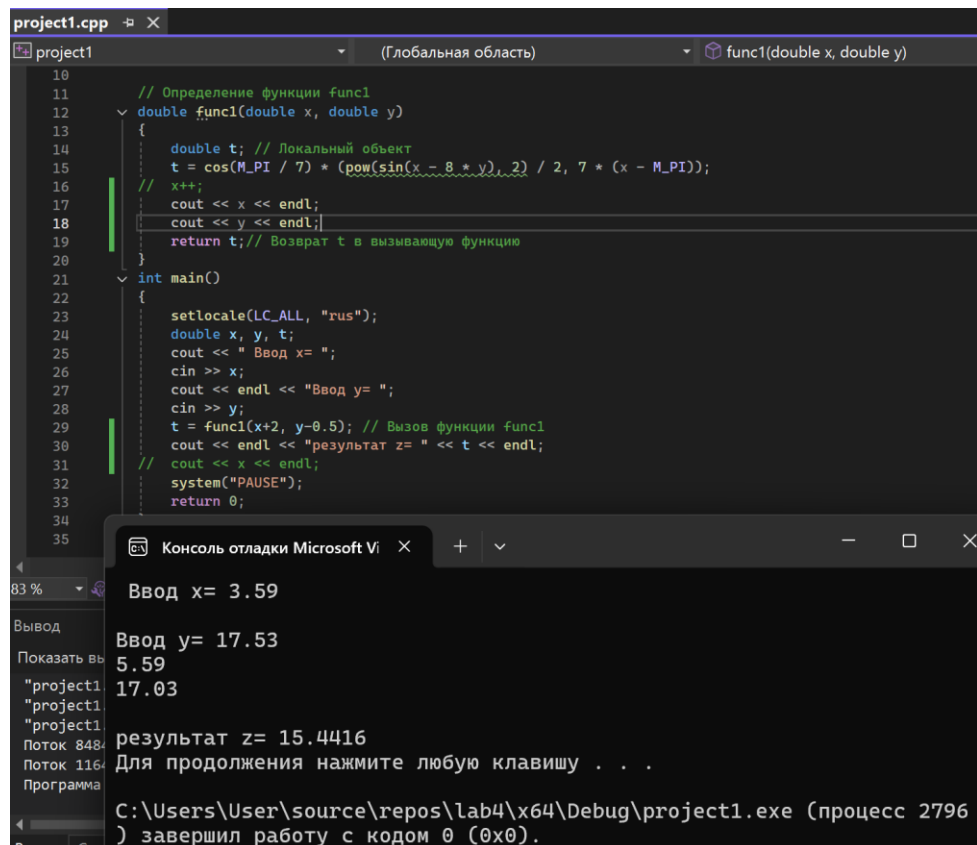
3.59

Рисунок 8. – результат работы при изменении значения x внутри функции

- проверим, можно ли при вызове функции в списке фактических параметров указывать не имя переменной, а константу или выражение, например, для следующих операторов:

`z = func1(x+2, y-0.5);`

Благодаря выводу внутри вызываемой функции мы понимаем, что переменные, поданные в неё, изменили свое значение, однако они остались неизменными в функции `main`. Результат изменился из-за измененных значений переменных, см. рисунок 9.



```
10
11 // Определение функции func1
12 double func1(double x, double y)
13 {
14     double t; // Локальный объект
15     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
16     // x++;
17     cout << x << endl;
18     cout << y << endl;
19     return t; // Возврат t в вызывающую функцию
20 }
21 int main()
22 {
23     setlocale(LC_ALL, "rus");
24     double x, y, t;
25     cout << " Ввод x= ";
26     cin >> x;
27     cout << endl << "Ввод y= ";
28     cin >> y;
29     t = func1(x+2, y-0.5); // Вызов функции func1
30     cout << endl << "результат z= " << t << endl;
31     // cout << x << endl;
32     system("PAUSE");
33     return 0;
34 }
35
```

Консоль отладки Microsoft Vi

Ввод x= 3.59

Ввод y= 17.53

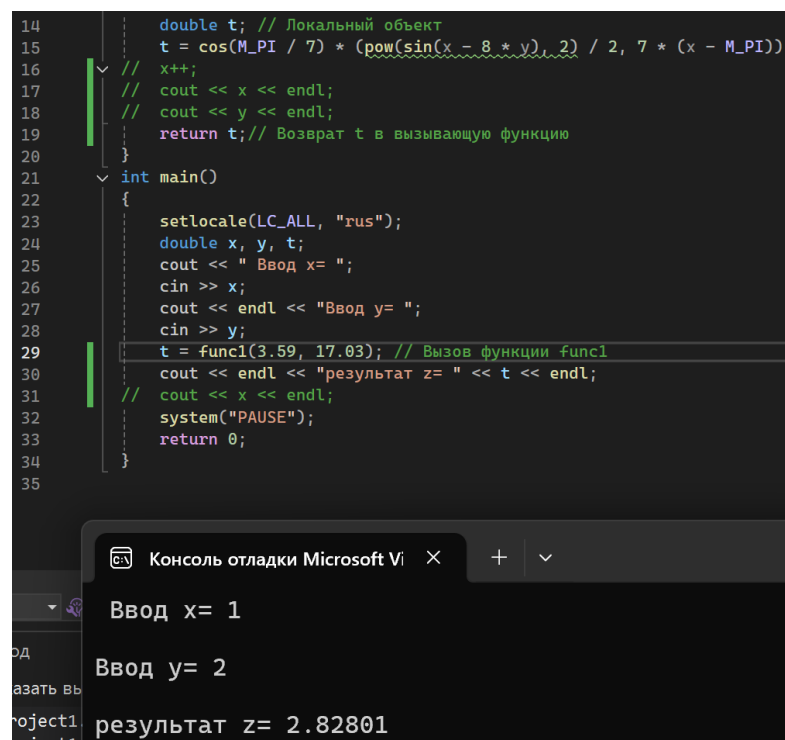
результат z= 15.4416

Для продолжения нажмите любую клавишу . . .

C:\Users\User\source\repos\lab4\x64\Debug\project1.exe (процесс 2796) завершил работу с кодом 0 (0x0).

Рисунок 9 – результат при выражении в списке параметров

$z = \text{func1}(3.59, 17.03)$; Указывая константы в качестве параметров при вызове функции, мы перестаем использовать переменные, вводимые в КОНСОЛЬ.



```
14 double t; // Локальный объект
15 t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
16 // x++;
17 // cout << x << endl;
18 // cout << y << endl;
19 return t; // Возврат t в вызывающую функцию
20 }
21 int main()
22 {
23     setlocale(LC_ALL, "rus");
24     double x, y, t;
25     cout << " Ввод x= ";
26     cin >> x;
27     cout << endl << "Ввод y= ";
28     cin >> y;
29     t = func1(3.59, 17.03); // Вызов функции func1
30     cout << endl << "результат z= " << t << endl;
31     // cout << x << endl;
32     system("PAUSE");
33     return 0;
34 }
35
```

Консоль отладки Microsoft Vi

Ввод x= 1

Ввод y= 2

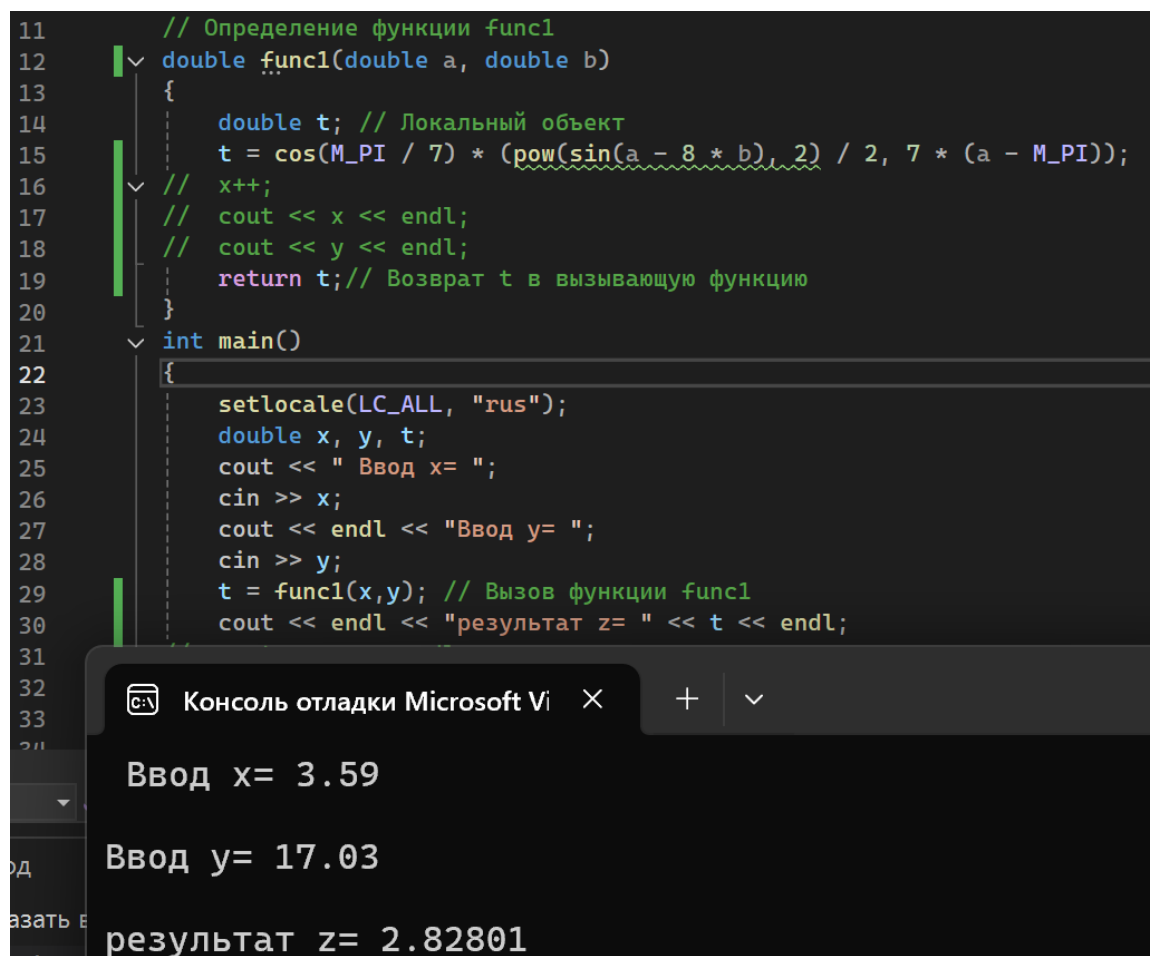
результат z= 2.82801

Рисунок 10. – результат при константе в списке параметров

- проверим, изменится ли результат работы проекта, если, ничего не меняя в главной функции main, изменить имена формальных параметров при определении функции func1 следующим образом:

```
// Определение функции func1  
double func1(double a, double b)
```

Как и в первом эксперименте, имена параметров используемых при вызове функции не обязаны соотноситься с переменными используемыми в самой функции. Изменение имен переменных внутри вызываемой функции не влияет на результат, см. рисунок 11.



```
11 // Определение функции func1  
12 double func1(double a, double b)  
13 {  
14     double t; // Локальный объект  
15     t = cos(M_PI / 7) * (pow(sin(a - 8 * b), 2) / 2, 7 * (a - M_PI));  
16     // x++;  
17     // cout << x << endl;  
18     // cout << y << endl;  
19     return t; // Возврат t в вызывающую функцию  
20 }  
21 int main()  
22 {  
23     setlocale(LC_ALL, "rus");  
24     double x, y, t;  
25     cout << " Ввод x= ";  
26     cin >> x;  
27     cout << endl << "Ввод y= ";  
28     cin >> y;  
29     t = func1(x,y); // Вызов функции func1  
30     cout << endl << "результат z= " << t << endl;  
31 }  
32  
33  
34
```

Консоль отладки Microsoft Visual Studio

Ввод x= 3.59

Ввод y= 17.03

результат z= 2.82801

Рисунок 11. Результат при изменении имен переменных внутри функции.

Реализация 2-го проекта

Создадим второй проект в уже существующем и назначим главным, см рисунок 12.

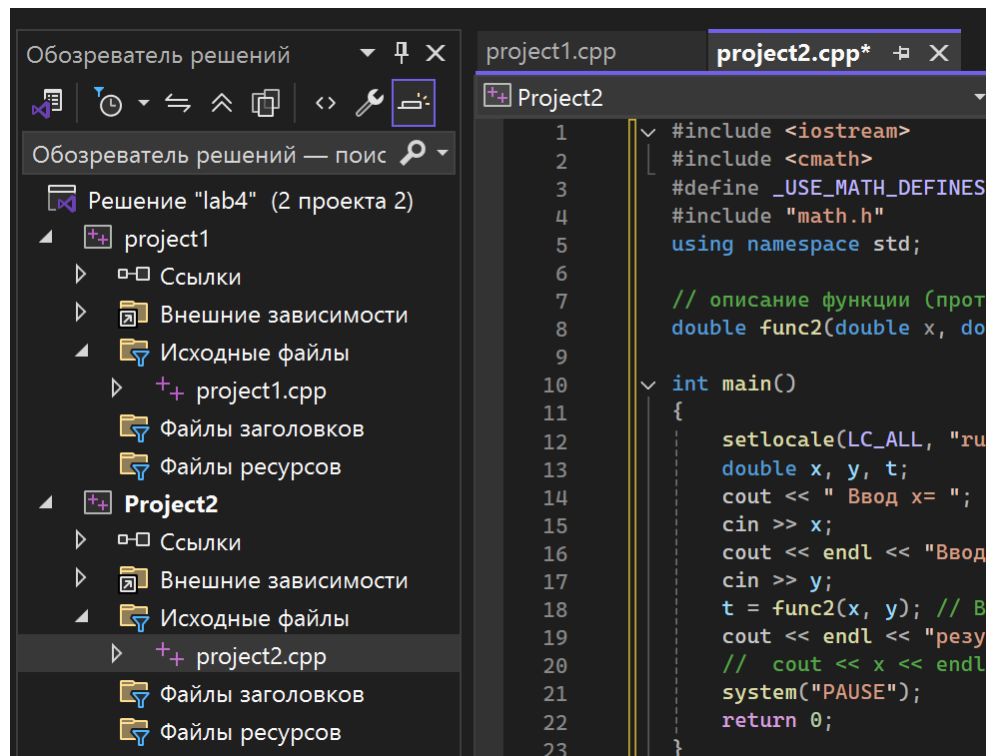


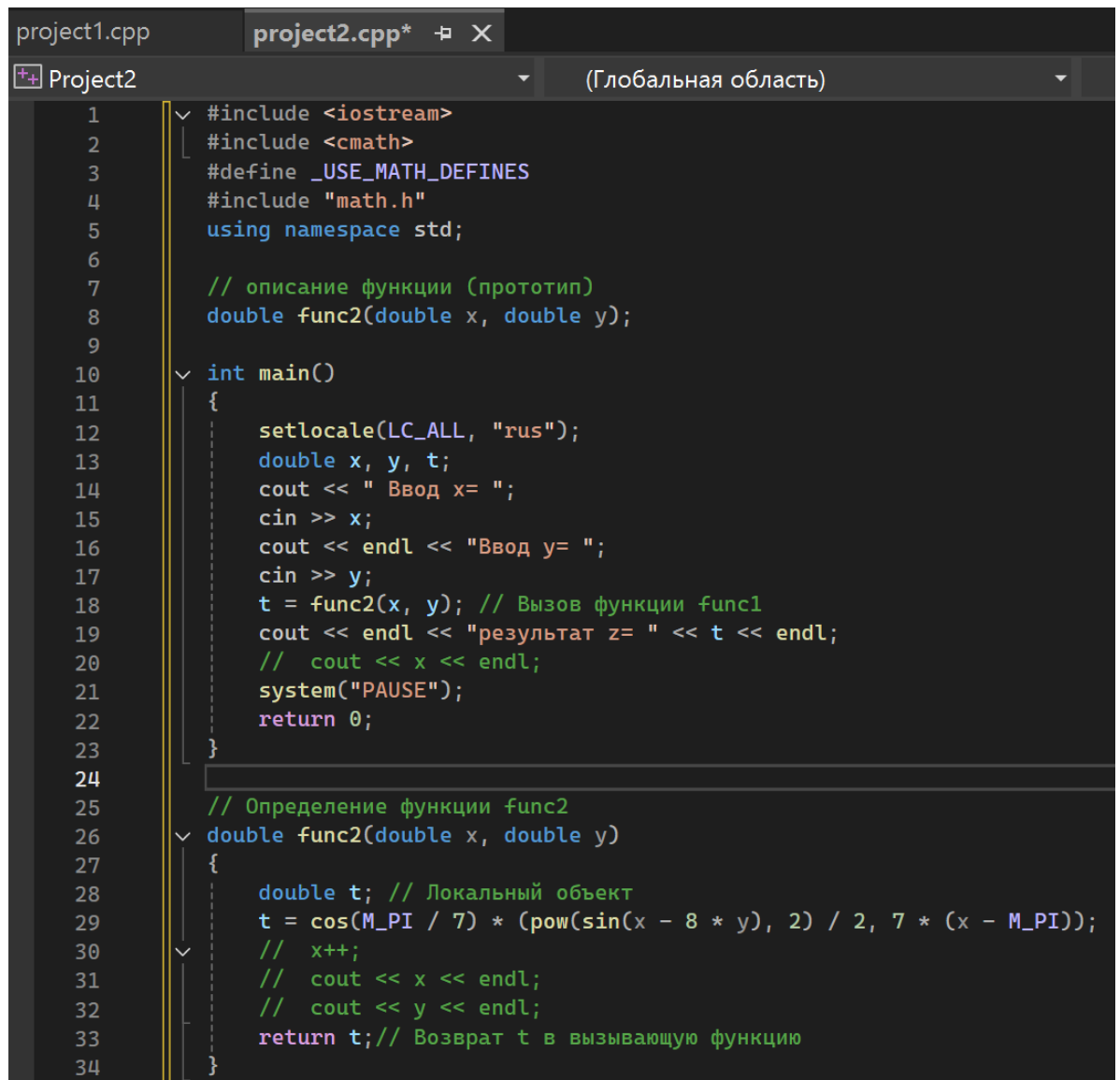
Рисунок 12. – создание второго проекта

Запишем в файл с именем `zad2.cpp` программные коды разработанных в первом проекте функций (переименовав функцию `func1` в `func2`) в следующем порядке:

- сначала программный код главной функции `main`;
- после него определение функции `func2` с параметрами и возвращаемым значением.

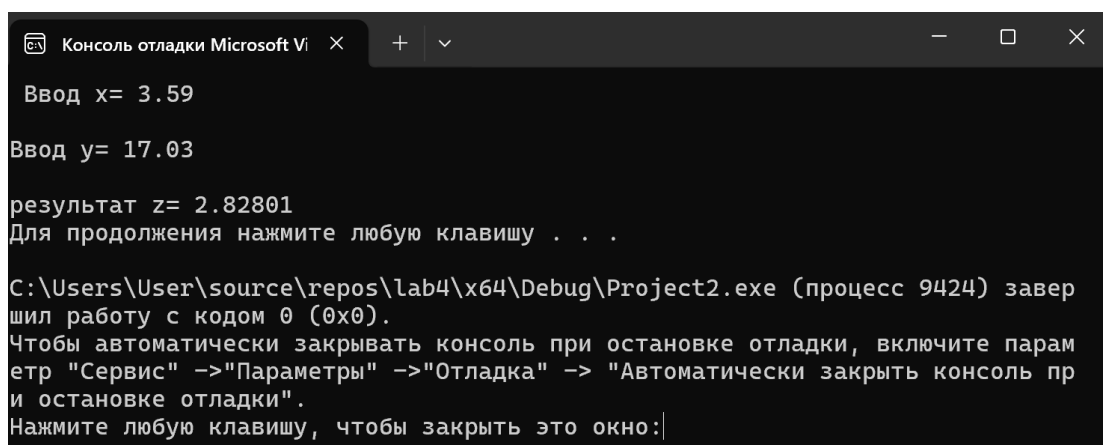
Перед кодом главной функции `main` запишем прототип функции `func2`, см рисунок 13.

Результат выполнения программы не изменился, см. рисунок 14.



```
1  #include <iostream>
2  #include <cmath>
3  #define _USE_MATH_DEFINES
4  #include "math.h"
5  using namespace std;
6
7  // описание функции (прототип)
8  double func2(double x, double y);
9
10 int main()
11 {
12     setlocale(LC_ALL, "rus");
13     double x, y, t;
14     cout << " Ввод x= ";
15     cin >> x;
16     cout << endl << "Ввод y= ";
17     cin >> y;
18     t = func2(x, y); // Вызов функции func1
19     cout << endl << "результат z= " << t << endl;
20     // cout << x << endl;
21     system("PAUSE");
22     return 0;
23 }
24
25 // Определение функции func2
26 double func2(double x, double y)
27 {
28     double t; // Локальный объект
29     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
30     // x++;
31     // cout << x << endl;
32     // cout << y << endl;
33     return t; // Возврат t в вызывающую функцию
34 }
```

Рисунок 13 – программный код проекта 2.

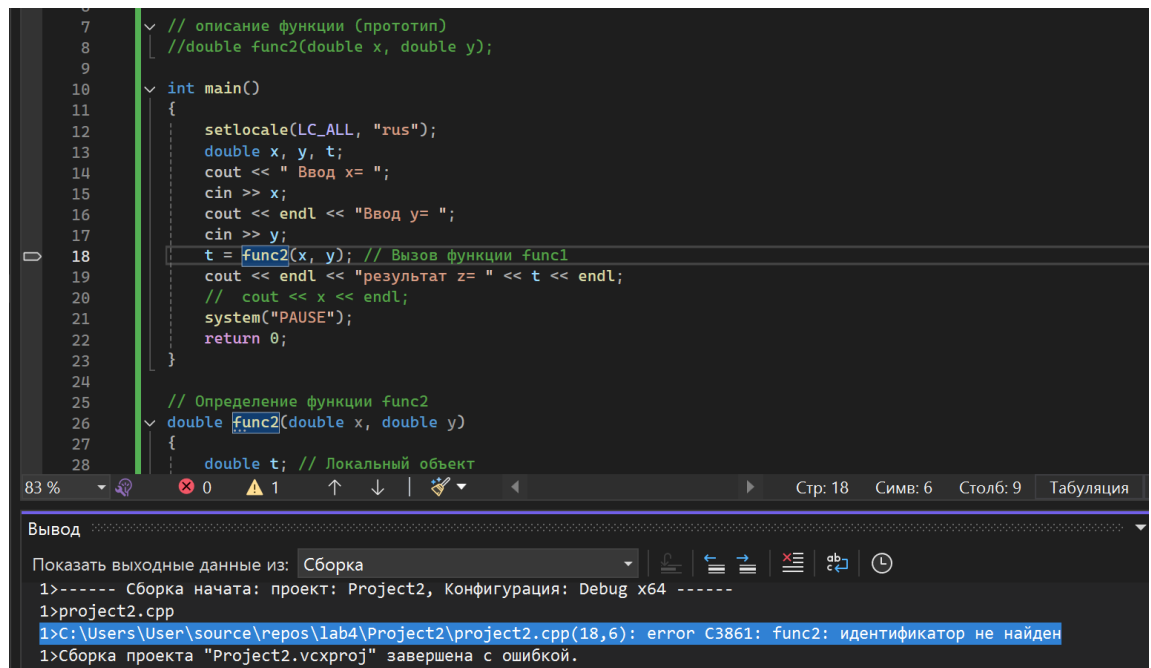


```
Консоль отладки Microsoft Vi
Ввод x= 3.59
Ввод y= 17.03
результат z= 2.82801
Для продолжения нажмите любую клавишу . . .
C:\Users\User\source\repos\lab4\x64\Debug\Project2.exe (процесс 9424) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 14. – результат выполнения проекта 2.

Проведем следующий эксперимент: закомментируем прототип функции func2 перед кодом главной функции main

Сборка завершена с ошибкой. Если заблаговременно не объявить функцию func2, функция main не будет знать к чему обращаться при её вызове.



```

7 // описание функции (прототип)
8 //double func2(double x, double y);
9
10 int main()
11 {
12     setlocale(LC_ALL, "rus");
13     double x, y, t;
14     cout << " Ввод x= ";
15     cin >> x;
16     cout << endl << "Ввод y= ";
17     cin >> y;
18     t = func2(x, y); // Вызов функции func1
19     cout << endl << "результат z= " << t << endl;
20     // cout << x << endl;
21     system("PAUSE");
22     return 0;
23 }
24
25 // Определение функции func2
26 double func2(double x, double y)
27 {
28     double t; // Локальный объект

```

Вывод

Показать выходные данные из: Сборка

1>----- Сборка начата: проект: Project2, Конфигурация: Debug x64 -----

1>project2.cpp

1>C:\Users\User\source\repos\lab4\Project2\project2.cpp(18,6): error C3861: func2: идентификатор не найден

1>Сборка проекта "Project2.vcxproj" завершена с ошибкой.

Рисунок 15. – сборка завершена с ошибкой

Реализация 3-го проекта

Создадим третий проект с именем pr3 в уже имеющемся решении. Разработаем алгоритм процедуры с параметрами и без возвращаемого значения. Схема алгоритма этой процедуры func3 представлена на рисунке 16.

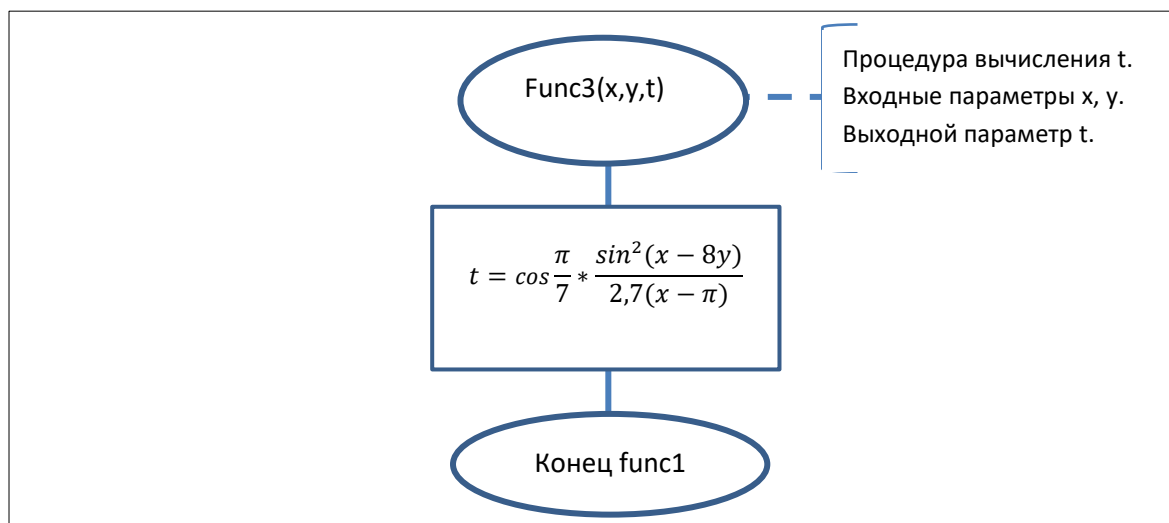
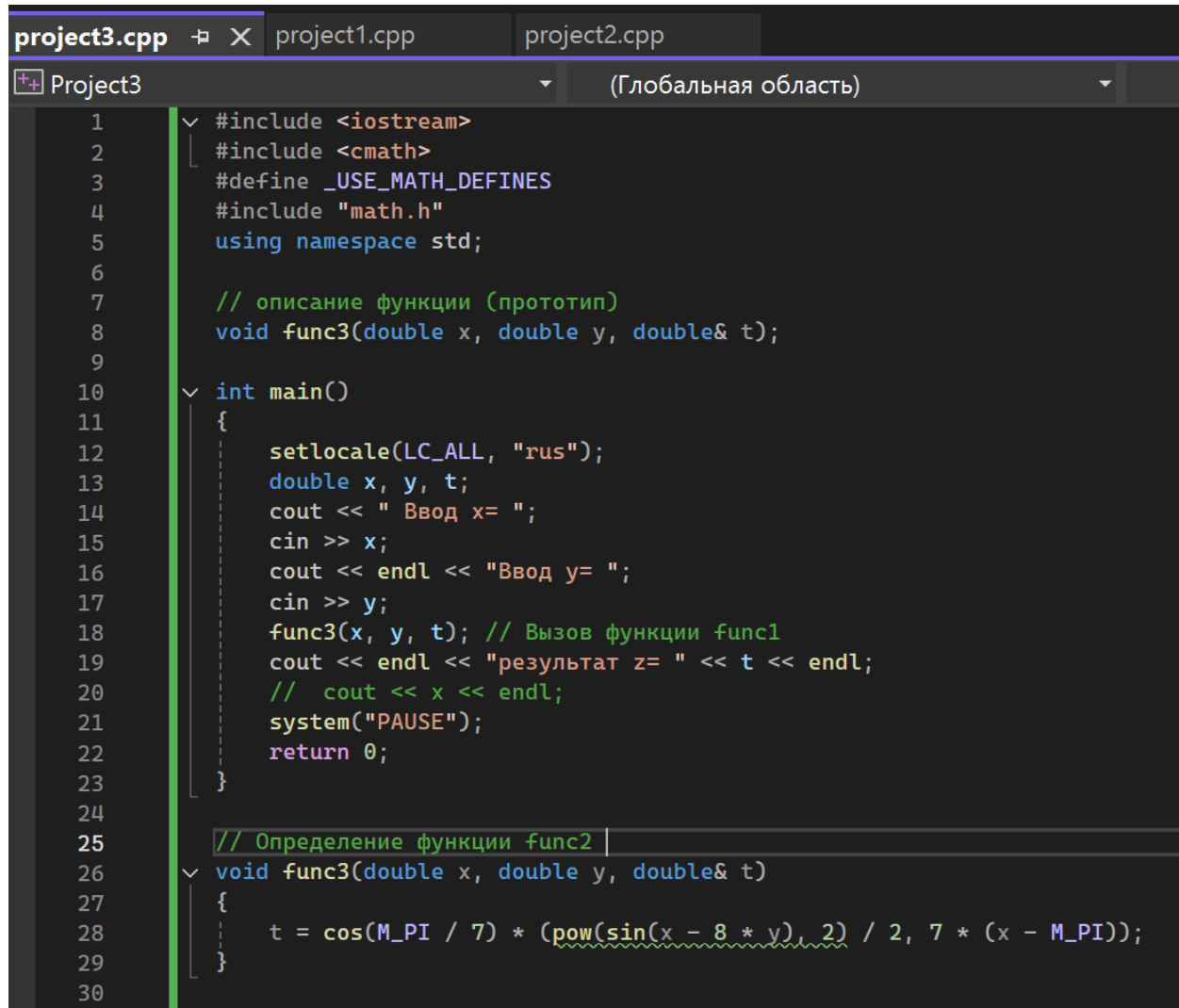


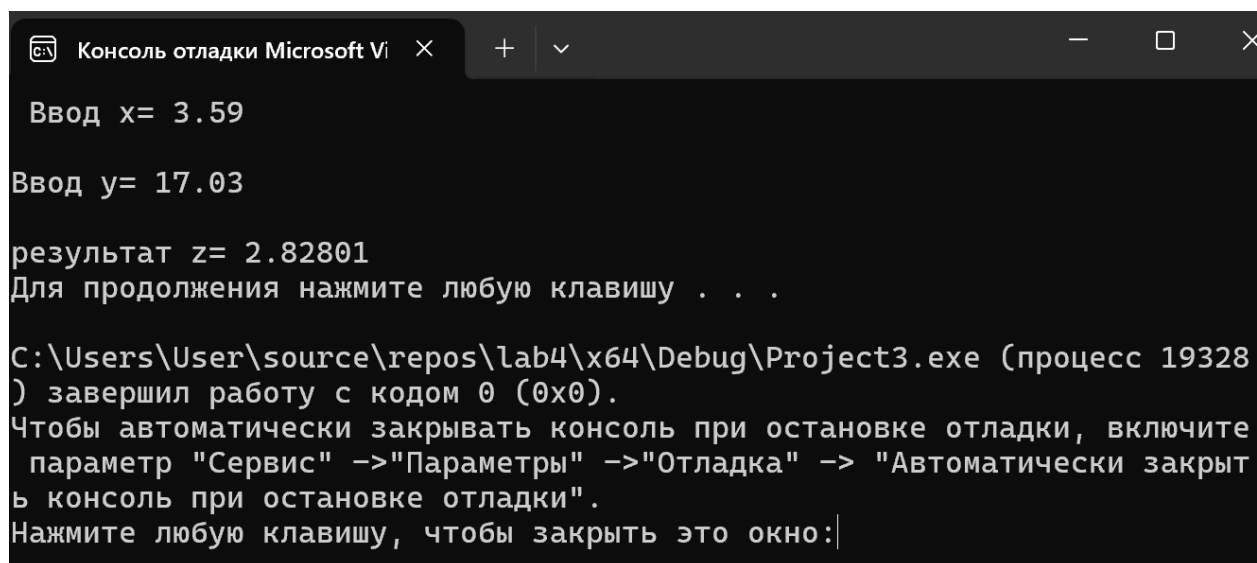
Рисунок 16 - Схема алгоритма процедуры func3 с параметрами и без возвращаемого значения для третьего проекта

Программный код третьего проекта представлен на рисунке 17, а результат выполнения – на рисунке 18.



```
1  #include <iostream>
2  #include <cmath>
3  #define _USE_MATH_DEFINES
4  #include "math.h"
5  using namespace std;
6
7  // описание функции (прототип)
8  void func3(double x, double y, double& t);
9
10 int main()
11 {
12     setlocale(LC_ALL, "rus");
13     double x, y, t;
14     cout << " Ввод x= ";
15     cin >> x;
16     cout << endl << "Ввод y= ";
17     cin >> y;
18     func3(x, y, t); // Вызов функции func1
19     cout << endl << "результат z= " << t << endl;
20     // cout << x << endl;
21     system("PAUSE");
22     return 0;
23 }
24
25 // Определение функции func2
26 void func3(double x, double y, double& t)
27 {
28     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
29 }
30
```

Рисунок 17 – программный код третьего проекта



```
Консоль отладки Microsoft Vi
Ввод x= 3.59
Ввод y= 17.03
результат z= 2.82801
Для продолжения нажмите любую клавишу . . .
C:\Users\User\source\repos\lab4\x64\Debug\Project3.exe (процесс 19328)
) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите
параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыт
ь консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 18 – результат выполнения третьего проекта

Проведем следующие эксперименты:

- проверим, можно ли при вызове функции в списке фактических параметров записывать выражения? Например:

`func3(x+2, y-0.5, z+1);` // Вызов функции `func3`

Ошибка возникает из-за несовместимости использованной структуры `double&` и выражения в качестве параметра, см. рисунок 19. Если выражения использовать только к `x` и `y`, которые типа `double`, проблем не будет, как и было проверено в одном из предыдущих экспериментов.

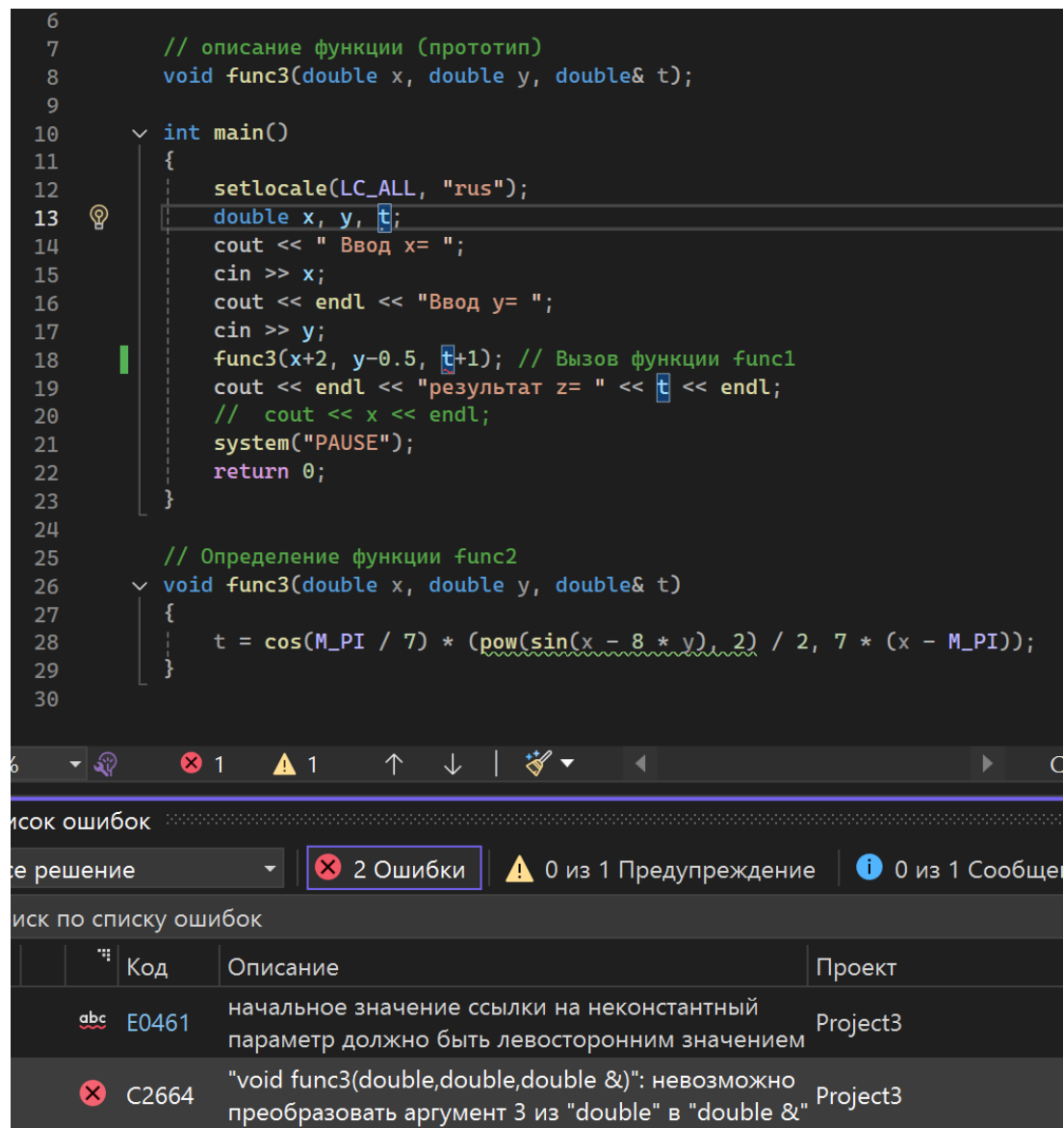


Рисунок 19 – ошибка преобразования `t`

- проверим, изменится ли результат выполнения проекта, если при определении функции func3 удалить знак & (операция взятия адреса) перед формальным параметром z:

```
void func3(double x, double y, double z)
```

Если при определении функции(и только там) удалить знак &, переменная t не будет определена, так как она будет считаться двумя разными переменными(одна double&, а другая просто double). Так что при вызове функции возникнет ошибка, см рисунок 20.

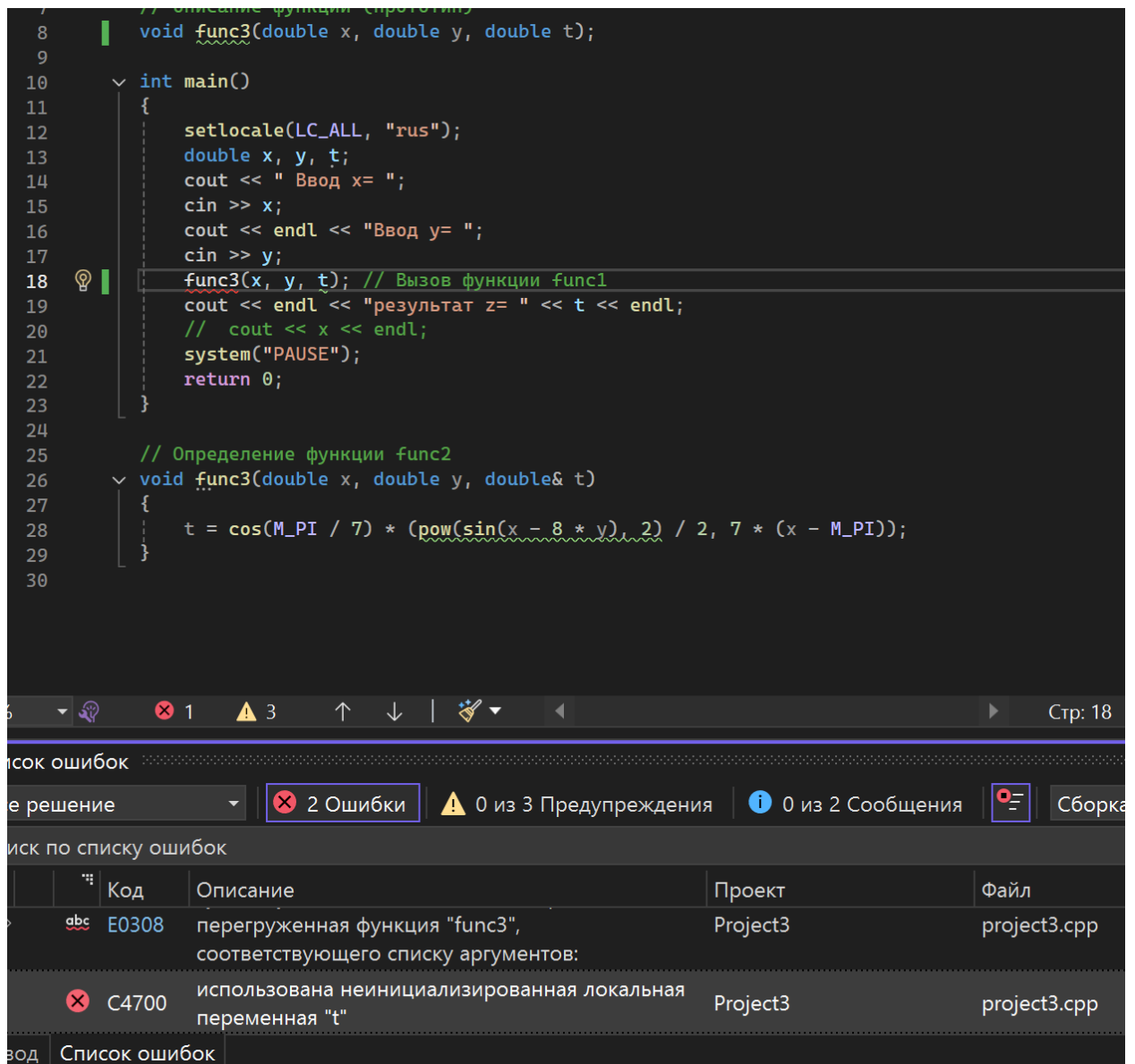


Рисунок 20 – неинициализированная переменная t

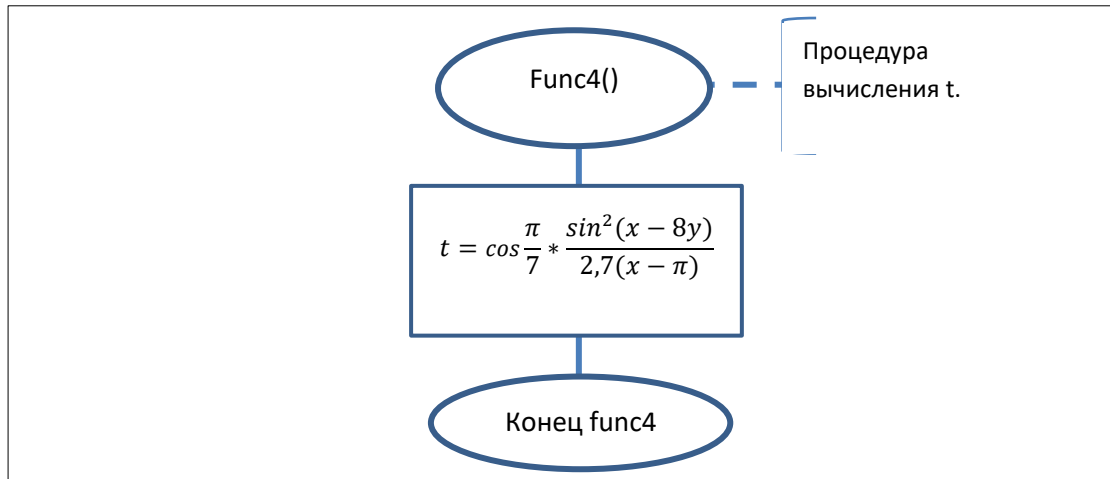
Без использования & переменная не может выйти за пределы функции в которой объявлена, это вызывает ошибку.

Реализация 4-го проекта

Создадим четвертый проект с именем pr4 в уже имеющемся решении.

Разработаем алгоритм процедуры без параметров и без возвращаемого значения. Схема алгоритма процедуры func4 представлена на рисунке 21.

Рисунок 21. – Схема алгоритма процедуры func4 без параметров и без

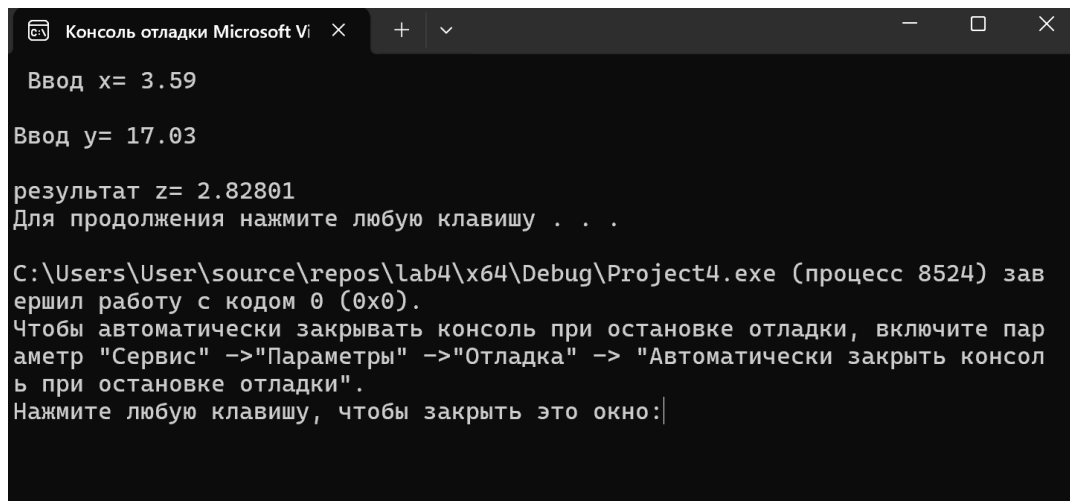


возвращаемого значения для четвертого проекта

Программный код проекта представлен на рисунке 22, а результат работы на рисунке 23.

```
1 #include <iostream>
2 #include <cmath>
3 #define _USE_MATH_DEFINES
4 #include "math.h"
5 using namespace std;
6
7 // описание функции (прототип)
8 void func4(void );
9 double x, y, t;
10 int main()
11 {
12     setlocale(LC_ALL, "rus");
13     cout << " Ввод x= ";
14     cin >> x;
15     cout << endl << "Ввод y= ";
16     cin >> y;
17     func4(); // Вызов функции func4
18     cout << endl << "результат z= " << t << endl;
19     // cout << x << endl;
20     system("PAUSE");
21     return 0;
22 }
23
24 // Определение функции func2
25 void func4()
26 {
27     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
28 }
```

Рисунок 22. – код программного проекта



```
Консоль отладки Microsoft Visual Studio
Ввод x= 3.59
Ввод y= 17.03
результат z= 2.82801
Для продолжения нажмите любую клавишу . . .

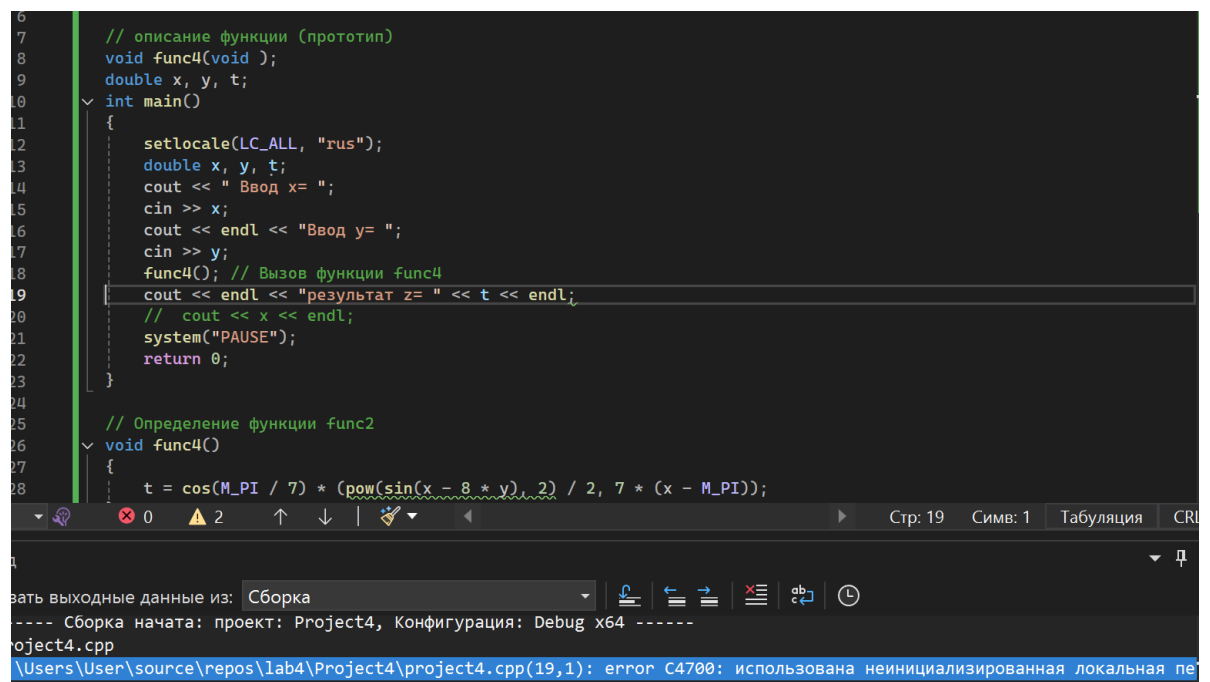
C:\Users\User\source\repos\lab4\x64\Debug\Project4.exe (процесс 8524) за-
вершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите пар-
аметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль
при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 23. – результат работы

Проведем следующий эксперимент:

проверим, изменятся ли результаты выполнения проекта, если в тело главной функции main добавить определение переменных x, y, z до оператора ввода исходных данных:

При выводе предпочтение отдается локальной переменной, которая не определена, что вызывает ошибку, см рисунок 24.



```
6 // описание функции (прототип)
7 void func4(void );
8 double x, y, t;
9 int main()
10 {
11     setlocale(LC_ALL, "rus");
12     double x, y, t;
13     cout << " Ввод x= ";
14     cin >> x;
15     cout << endl << "Ввод y= ";
16     cin >> y;
17     func4(); // Вызов функции func4
18     cout << endl << "результат z= " << t << endl;
19     // cout << x << endl;
20     system("PAUSE");
21     return 0;
22 }
23
24 // Определение функции func2
25 void func4()
26 {
27     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
28 }
```

Сборка
---- Сборка начата: проект: Project4, Конфигурация: Debug x64 ----
project4.cpp
C:\Users\User\source\repos\lab4\Project4\project4.cpp(19,1): error C4700: использована неинициализированная локальная переменная 't' [Project4.sln]

Рисунок 24 – ошибка локальной переменной

Реализация 5-го проекта

Создадим в уже имеющемся решении пятый проект с именем project5, состоящий из двух файлов. В первый файл с именем project5_main.cpp

поместим текст функции main из второго проекта. Во второй файл с именем project5_func.cpp поместим текст функции func2 из того же проекта, см. рисунок 25.

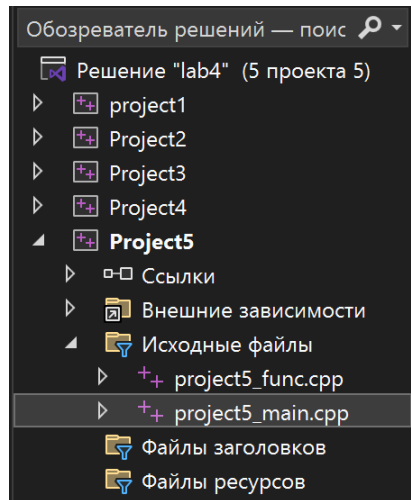
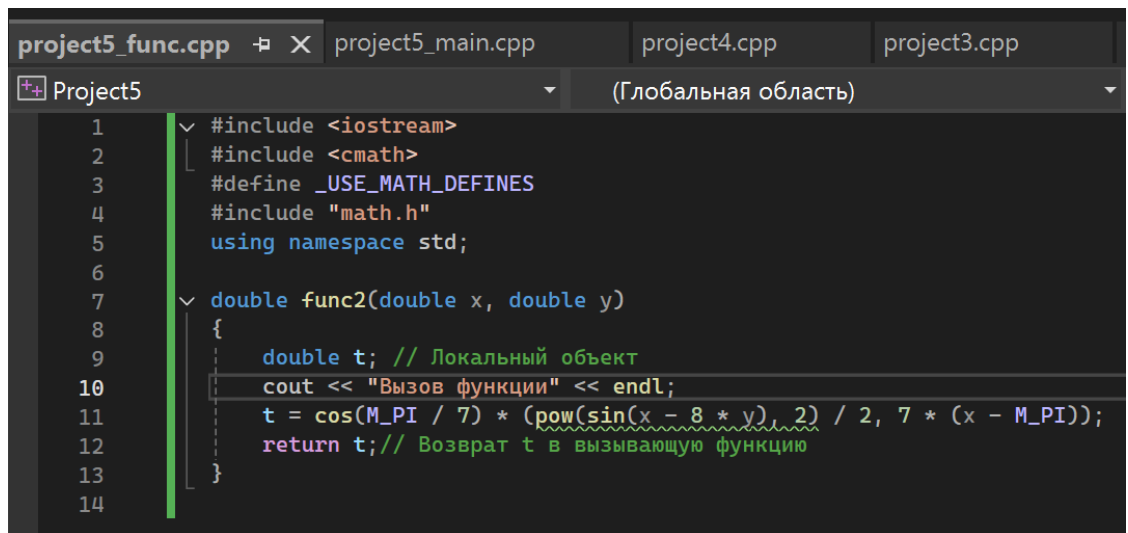


Рисунок 25. – файлы пятого проекта

После отдельной компиляции файлов выполним их совместную компоновку (перестроение решения) и выполнение проекта. Программный код приведен на рисунках 26 и 27. Результаты выполнения при заданных значениях исходных данных приведены на рисунке 28.

```
project5_func.cpp  project5_main.cpp  project4.cpp  project3
Project5 (Глобальная область)
1  #include <iostream>
2  #include <cmath>
3  #define _USE_MATH_DEFINES
4  #include "math.h"
5  using namespace std;
6
7  // описание функции (прототип)
8  double func2(double x, double y);
9
10 int main()
11 {
12     setlocale(LC_ALL, "rus");
13     double x, y, t;
14     cout << " Ввод x= ";
15     cin >> x;
16     cout << endl << "Ввод y= ";
17     cin >> y;
18     t = func2(x, y); // Вызов функции func1
19
20     cout << endl << "результат z= " << t << endl;
21     // cout << x << endl;
22     system("PAUSE");
23     return 0;
24 }
25
```

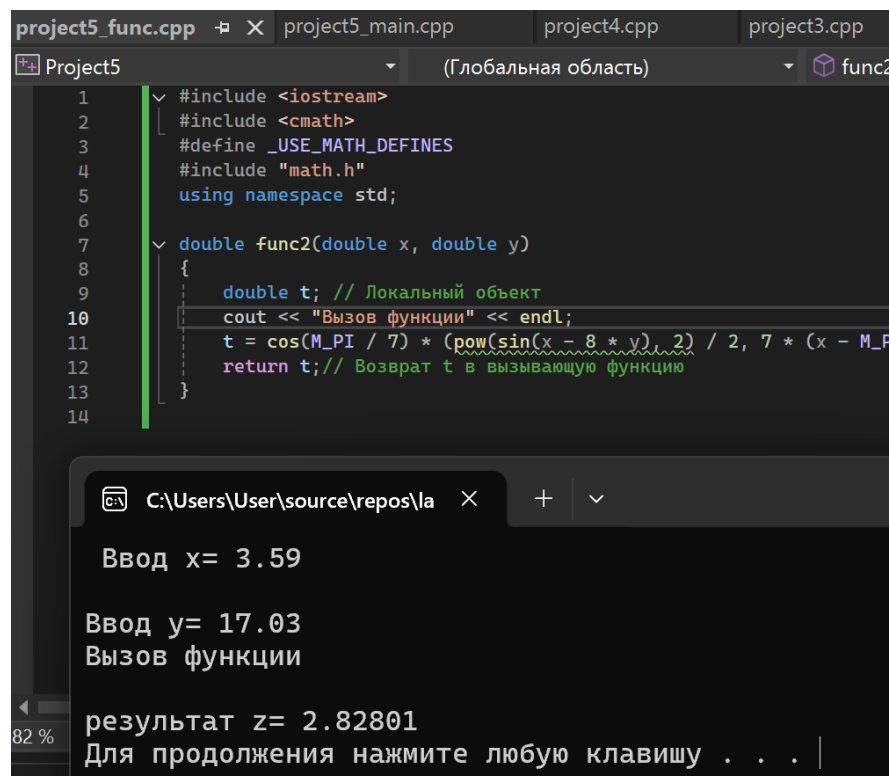
Рисунок 26 – программный код функции main



```
1  #include <iostream>
2  #include <cmath>
3  #define _USE_MATH_DEFINES
4  #include "math.h"
5  using namespace std;
6
7  double func2(double x, double y)
8  {
9      double t; // Локальный объект
10     cout << "Вызов функции" << endl;
11     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
12     return t; // Возврат t в вызывающую функцию
13 }
14
```

Рисунок 27 – программный код вызываемой функции

В качестве дополнительной проверки была добавлена строка, выводящаяся только при вызове функции.



```
project5_func.cpp  X project5_main.cpp  project4.cpp  project3.cpp
Project5  (Глобальная область)  func2
1  #include <iostream>
2  #include <cmath>
3  #define _USE_MATH_DEFINES
4  #include "math.h"
5  using namespace std;
6
7  double func2(double x, double y)
8  {
9      double t; // Локальный объект
10     cout << "Вызов функции" << endl;
11     t = cos(M_PI / 7) * (pow(sin(x - 8 * y), 2) / 2, 7 * (x - M_PI));
12     return t; // Возврат t в вызывающую функцию
13 }
14

C:\Users\User\source\repos\la  X  +  v
Ввод x= 3.59
Ввод y= 17.03
Вызов функции
результат z= 2.82801
Для продолжения нажмите любую клавишу . . . |
```

Рисунок 28 – результат выполнения программы

4) Доказательство правильности результатов

Результат выполнения всех пяти проектов одинаков и равен 2.82801. Выполним расчет арифметического выражения с использованием калькулятора или программы Microsoft Excel и получим совпадающий результат, что доказывает его правильность.